SIMATS Engineering
Department of Information Technology
MLA04 - DEEP LEARNING
ASSIGNMENT 2

CO2 – DEEP LEARNING ARCHITECTURES, REPRESENTATION LEARNING AND UNSUPERVISED NEURAL NETWORKS

Nature: Individual

Duration: 1 Week

Total Marks: 50

Name: D. CHAITANYA KUMAR REDDY

Register No: 192411094

CO2: Apply and analyze deep learning architectures, representation learning, the effect of network width and depth, activation functions (ReLU, LReLU and ERELU), unsupervised training of neural networks, Restricted Boltzmann Machines, auto encoders and deep learning applications to make justified architectural and training decisions.

Topics: Machine Learning and Deep Learning, Representation Learning, Width and Depth of Neural Networks, Activation Functions – ReLU, LReLU, ERELU, Unsupervised Training of Neural Networks, Restricted Boltzmann Machines, Auto Encoders, Deep Learning Applications.

Purpose

Investigate how representation learning, network width and depth, activation behaviour and unsupervised models such as Restricted Boltzmann Machines and auto encoders influence the reliability of deep learning systems. Students must connect equations, architectures and numerical behaviour to practical design decisions.

Scenario: Smart Hospital Chest X-ray Screening

A hospital network plans to build an intelligent screening system that analyses chest X-ray images and flags suspicious scans for a radiologist. Each image is downsampled to a 28 × 28 grayscale patch (784 pixel values). The network holds 100,000 unlabelled scans but only 1,000 scans labelled by radiologists as "normal" or "abnormal". It must decide which architecture, activation function, unsupervised training method and application strategy to use. You are required to analyze the options, select suitable techniques and justify every decision using mathematical reasoning.

TASK 1 – Machine Learning and Deep Learning | 3 Marks

The hospital currently classifies scans with a classical machine-learning pipeline: engineers hand design features such as intensity histograms and edge counts, and a support vector machine classifies them. A deep neural network that learns directly from pixels is now being proposed.

a) Differentiate machine learning and deep learning in terms of feature extraction, data requirement, computational cost and interpretability.

Ans: Comparison of machine learning (ML) and deep learning (DL):

Aspect	Machine learning (hand-designed features + SVM)	Deep learning (neural network on pixels)
Feature extraction	Manual: engineers design features such as intensity histograms and edge counts	Automatic: features are learned from raw pixels, layer by layer, together with the classifier

Data requirement	Works with small or medium labelled data (e.g. 1,000 scans)	Needs large data (thousands to millions of examples) or pre-training to avoid overfitting
Computational cost	Low: cheap training and inference, CPU is enough	High: millions of parameters, long training, usually GPU
Interpretability	Higher: features are human-meaningful and the SVM margin is easy to analyse	Lower: a "black box"; needs tools such as saliency maps / Grad-CAM to explain decisions

b) Explain why the classical pipeline may fail to capture subtle lesion patterns in raw X-ray images, and how a deep network addresses this limitation.

Ans: Hand-designed features squeeze an image into a few global statistics. A histogram throws away **where** the intensities occur and an edge count ignores shape and context, so small, low-contrast lesions or subtle texture changes are lost before the SVM even sees the data, and the SVM can only use what the engineers thought of in advance.

A deep (convolutional) network learns a hierarchy of filters directly from the pixels and optimises them for the diagnosis task: edges → textures → anatomical structures → lesion patterns. Because it keeps spatial relationships and is trained end-to-end, it can discover subtle patterns that no one hand-coded.

c) With only 1,000 labelled scans, state one risk of training a deep network from scratch and one way to reduce that risk.

Ans: Risk: overfitting. A deep network has very many parameters (10^5–10^6 or more) compared with 1,000 labelled scans, so it can memorise the training scans (near-zero training error) and generalise poorly to new patients.

Mitigation: use the 100,000 unlabelled scans for unsupervised pre-training (auto encoder/RBM) or transfer learning, then fine-tune on the 1,000 labels; data augmentation, dropout/weight decay and early stopping also reduce the risk.

Evidence Expected: *Clear comparison, reasoning on feature learning, and a justified risk-mitigation strategy.*

TASK 2 – Representation Learning | 4 Marks

A raw scan has 784 pixel values, yet the normal/abnormal decision depends on a few high-level patterns. The following toy data shows why the representation of the data matters. Four points (x_1, x_2) are given: (0, 0) and (1, 1) belong to class A; (0, 1) and (1, 0) belong to class B.

a) Define representation learning and explain why raw pixel values are a poor representation for classification.

Ans: Representation learning is learning, from data, a transformation $\varphi(x)$ of the raw input into features that make the task (here normal vs abnormal) easy, instead of designing the features by hand.

Raw pixels are a poor representation because: they are very high-dimensional (784) and strongly correlated; a small shift, rotation or change in brightness changes many pixel values although the diagnosis is the same; the class information lies in a few high-level patterns (shape, texture, location of opacities) which are not visible in individual pixel values, and a linear/SVM classifier on raw pixels cannot capture them.

b) Show that no single straight line in the (x_1, x_2) space can separate class A from class B.

Ans: Suppose a line $w_1x_1 + w_2x_2 + b = 0$ puts class A on the positive side and class B on the negative side. Then:

$(0,0) \in A$: $b > 0$; $(1,1) \in A$: $w_1 + w_2 + b > 0$; $(0,1) \in B$: $w_2 + b < 0$; $(1,0) \in B$: $w_1 + b < 0$.

Adding the last two inequalities gives $w_1 + w_2 + 2b < 0$. But from the first two, $w_1 + w_2 + b > 0$ and $b > 0$ give $w_1 + w_2 + 2b > 0$. This is a **contradiction**, so no single straight line separates A from B (the XOR problem). (The opposite orientation gives the same contradiction.)

c) Using the new representation $\varphi(x) = (x_1, x_2, x_1x_2)$, compute φ for the four points and show that $f = x_1 + x_2 - 2x_1x_2$ separates the two classes linearly in the new space.

Ans: $\varphi(x) = (x_1, x_2, x_1x_2)$: $(0,0) \rightarrow (0, 0, 0)$; $(1,1) \rightarrow (1, 1, 1)$; $(0,1) \rightarrow (0, 1, 0)$; $(1,0) \rightarrow (1, 0, 0)$.

$f = x_1 + x_2 - 2x_1x_2$: $(0,0) \rightarrow 0$; $(1,1) \rightarrow 1 + 1 - 2 = 0$; $(0,1) \rightarrow 1$; $(1,0) \rightarrow 1$.

So class A gives $f = 0$ and class B gives $f = 1$. In the new space f is a **linear** function (weights 1, 1, −2) of φ, so the plane $\varphi_1 + \varphi_2 - 2\varphi_3 = 0.5$ separates the two classes perfectly. A good representation turned a non-linearly separable problem into a linearly separable one.

d) Describe the hierarchy of features learned by successive layers of a deep network on chest X-ray images.

Ans: Successive layers learn increasingly abstract features of a chest X-ray: **layer 1** – edges, intensity gradients and blobs; **layer 2** – textures and simple shapes (rib edges, vessel lines, lung borders); **layer 3** – parts and structures (lung fields, heart outline, nodules, opacities); **deeper layers** – whole-scan patterns such as consolidation or an effusion that separate abnormal from normal. Each layer builds on the one below it.

Evidence Expected: *Definition, proof of non-separability, computed representation and explanation of the feature hierarchy.*

TASK 3 – Width and Depth of Neural Networks | 4 Marks

Two candidate architectures take the 784-pixel input and produce 10 output scores. Network W (wide and shallow) has one hidden layer of 1,000 neurons. Network D (deep and narrow) has three hidden layers of 128 neurons each. Every layer is fully connected and includes bias terms.

a) Calculate the number of trainable parameters in Network W and in Network D.

Ans: Network W (784–1000–10): hidden layer $784 \times 1000 + 1000 = 785{,}000$; output layer $1000 \times 10 + 10 = 10{,}010$. **Total = 795,010 parameters.**

Network D (784–128–128–128–10): layer 1: $784 \times 128 + 128 = 100{,}480$; layer 2: $128 \times 128 + 128 = 16{,}512$; layer 3: 16,512; output: $128 \times 10 + 10 = 1{,}290$. **Total = 134,794 parameters.**

b) Explain the difference between the width and the depth of a network, and state what the universal approximation theorem says about a wide single-hidden-layer network.

Ans: Width is the number of neurons in a layer; **depth** is the number of layers (sequential non-linear transformations). W is wide and shallow; D is narrow and deep.

Universal approximation theorem: a feed-forward network with a single hidden layer and enough neurons with a non-linear activation can approximate any continuous function on a compact domain to any accuracy. It guarantees that such a function **exists**, but does not say how many neurons are needed (possibly exponentially many), that training will find it, or that it will generalise.

c) Explain why a deep network can represent some functions with far fewer neurons than a shallow network, and name one training difficulty that increases with depth.

Ans: A deep network composes simple features into more complex ones and **reuses** them, so the number of linear regions / functional complexity grows exponentially with depth but only

polynomially with width. Some functions (e.g. parity of n bits, or hierarchical image patterns) need O(n) neurons with depth but exponentially many neurons in one hidden layer. Hence deep networks are far more parameter-efficient for hierarchical data such as images.

Training difficulty that grows with depth: vanishing (or exploding) gradients; deeper networks are also harder to optimise.

d) Recommend one of the two networks for 1,000 labelled scans and justify the choice using parameter count and generalisation.

Ans: Recommendation: Network D. It has about 5.9 times fewer parameters (134,794 vs 795,010), i.e. about 135 parameters per labelled scan instead of about 795, so it has lower variance and is less prone to memorising 1,000 scans. Its layered structure learns reusable hierarchical features, which generalises better, and it is cheaper in memory and computation. With ReLU-type activations and regularization (dropout, weight decay, early stopping), the depth of 3 is manageable. Network W has high capacity but would overfit badly with so few labels.

Evidence Expected: *Parameter counts, width versus depth explanation, efficiency argument and a justified recommendation.*

TASK 4 – Activation Functions: ReLU, LReLU and ERELU | 5 Marks

A hidden neuron receives the pre-activation values $z = -3, -1, 0$ and 2. Use ReLU, $f(z) = \max(0, z)$; Leaky ReLU (LReLU), $f(z) = z$ for $z > 0$ and $0.01z$ for $z \leq 0$; and ERELU (exponential rectified linear unit), $f(z) = z$ for $z > 0$ and $f(z) = \alpha(e^z - 1)$ for $z \leq 0$ with $\alpha = 1$.

a) Compute the output of ReLU, LReLU and ERELU for each of the four inputs, correct to four decimal places.

Ans: Outputs ($\alpha = 1$ for ERELU, ERELU(z) = e^z − 1 for $z \leq 0$):

z	ReLU	LReLU	ERELU
−3	0.0000	−0.0300	−0.9502
−1	0.0000	−0.0100	−0.6321
0	0.0000	0.0000	0.0000
2	2.0000	2.0000	2.0000

b) Compute the derivative of each activation function at $z = -3, -1$ and 2.

Ans: Derivatives (ReLU: 1 for $z > 0$ else 0; LReLU: 1 for $z > 0$ else 0.01; ERELU: 1 for $z > 0$ else $\alpha \cdot e^z$):

z	ReLU′	LReLU′	ERELU′
−3	0.0000	0.0100	0.0498
−1	0.0000	0.0100	0.3679
2	1.0000	1.0000	1.0000

c) Compare the three functions in terms of zero-centred output, smoothness, computational cost and behaviour for negative inputs.

Property	ReLU	LReLU	ERELU
Zero-centred output	No: outputs ≥ 0, mean is positive	Almost no: tiny negative values	Closest: negative outputs (down to −α) pull the mean towards zero

Smoothness	Kink at $z = 0$ (not differentiable)	Kink at $z = 0$ (not differentiable)	Smooth; with $\alpha = 1$ the slope is 1 on both sides of 0 (continuous derivative)
Computational cost	Lowest (one comparison)	Very low (comparison + one multiply)	Highest (needs an exponential for $z \leq 0$)
Negative inputs	Output 0 and gradient 0 (neuron is inactive)	Small linear leak $0.01z$, gradient 0.01	Saturates smoothly to $-\alpha$; gradient αe^z decays towards 0 for very negative z

d) State which function is most likely to produce neurons that stop learning and explain why.

Ans: ReLU is the most likely to produce "dead" neurons. For every $z \leq 0$ its output and its gradient are exactly zero, so if a neuron's pre-activation becomes negative for all inputs (e.g. after a large update) no gradient flows back, its weights stop changing and it can never recover. LReLU (slope 0.01) and ERELU (gradient $e^z > 0$) always keep a non-zero gradient, so they avoid this.

Evidence Expected: *Numerical outputs, derivative values, a structured comparison and a justified conclusion.*

TASK 5 – Dying Neurons and Choice of Activation | 3 Marks

During training with a very large learning rate, a ReLU neuron with two inputs reaches the weights $w = (1, 1)$ and bias $b = -3$. The input features are scaled to the range $[0, 1]$.

a) Calculate the largest possible pre-activation z and show that the neuron outputs zero for every input.

Ans: $z = w_1x_1 + w_2x_2 + b = x_1 + x_2 - 3$ with $x_1, x_2 \in [0, 1]$. The largest value occurs at $x = (1, 1)$: $z_max = 1 + 1 - 3 = \mathbf{-1}$.

Since $z \leq -1 < 0$ for every possible input, $\text{ReLU}(z) = \max(0, z) = \mathbf{0}$ for all inputs, so the neuron is dead.

b) Explain, using the gradient, why such a neuron cannot recover during further training.

Ans: By the chain rule, $\partial L/\partial w = (\partial L/\partial h) \cdot f'(z) \cdot x$ and $\partial L/\partial b = (\partial L/\partial h) \cdot f'(z)$, with $f'(z) = 0$ for $z < 0$ for ReLU. As $z < 0$ for every input, both gradients are exactly zero whatever the loss is, so the update $w \leftarrow w - \eta \cdot 0$ leaves $w = (1, 1)$ and $b = -3$ unchanged. The neuron keeps outputting zero and the gradient stays zero, so it cannot recover.

c) For $z = -1$, compare the gradient passed back by ReLU, LReLU (slope 0.01) and ERELU ($\alpha = 1$), and suggest two other measures to avoid the problem.

Ans: At $z = -1$: **ReLU′ = 0**; **LReLU′ = 0.01**; **ERELU′ = $\alpha \cdot e^{-1}$ = 0.3679**. So ReLU passes no gradient, LReLU a very small one and ERELU a substantial one.

Two other measures: (1) use a smaller learning rate (or learning-rate warm-up/decay and gradient clipping) so that one big update cannot push the weights/bias into the dead region; (2) use He initialization together with batch normalization (or a small positive bias), which keeps pre-activations centred around zero so that the neuron stays active for some inputs.

Evidence Expected: *Calculation, gradient-based reasoning, numerical comparison and justified remedies.*

TASK 6 – Unsupervised Training of Neural Networks | 3 Marks

The hospital has 100,000 unlabelled scans and only 1,000 labelled scans. The team plans to use the unlabelled data to help the network learn useful features before the labels are used.

a) Define unsupervised training of a neural network and state what the training objective looks like when no labels are available.

Ans: Unsupervised training trains a network using only the inputs x (no labels y), so that it discovers structure in the data. The objective contains no label term: for an auto encoder it is the reconstruction error $\min_\theta (1/N) \Sigma \|x - g(f(x))\|^2$; for an RBM it is maximising the data likelihood $\max_\theta \Sigma \log p_\theta(v)$ (equivalently minimising the energy of the training data).

b) Explain greedy layer-wise unsupervised pre-training followed by supervised fine-tuning, listing the steps in order.

Ans: Greedy layer-wise pre-training followed by fine-tuning:

1. Train the first layer (an RBM or auto encoder) on the 100,000 unlabelled scans so that it models the 784 raw inputs; this gives hidden features h^1.

2. Freeze layer 1, compute h^1 for all scans and train a second layer on h^1 to obtain h^2.

3. Repeat for further layers, one at a time (greedy).

4. Stack the pre-trained layers into one deep network and add an output layer (normal/abnormal, sigmoid or softmax) on top.

5. Fine-tune the whole network with supervised back-propagation on the 1,000 labelled scans, using a small learning rate.

c) Explain two benefits of unsupervised pre-training when labelled data is scarce.

Ans: Benefit 1 – better features from scarce labels: the 100,000 unlabelled scans teach the network general X-ray structure, so only a few labels are needed to learn the decision, which lowers overfitting and improves generalization (the pre-training acts as a regularizer).

Benefit 2 – a good starting point for optimisation: the pre-trained weights start in a good region of the parameter space instead of random values, which speeds up supervised training, helps avoid poor local minima and weakens vanishing-gradient problems in deep networks.

Evidence Expected: *Definition, ordered training steps and justified benefits.*

TASK 7 – Restricted Boltzmann Machines: Energy and Inference | 5 Marks

An RBM has 3 binary visible units v and 2 binary hidden units h with energy $E(v, h) = -a^Tv - b^Th - v^TWh$. The visible biases are a = (0.1, 0, −0.1) and the hidden biases are b = (−0.5, 0.2). The weight matrix W, with rows for the visible units and columns for the hidden units, is W = [[1.0, −0.5], [0.5, 1.0], [−1.0, 0.5]]. Consider the visible vector v = (1, 0, 1).

a) Write the joint probability P(v, h) in terms of the energy and explain the role of the partition function Z.

Ans: $P(v, h) = e^{(-E(v, h))} / Z$, where $Z = \Sigma_{v,h} e^{(-E(v, h))}$.

The partition function Z is the normalising constant that makes all probabilities sum to 1 (low energy ⇒ high probability). It requires a sum over all $2^{(3+2)} = 32$ joint states here and $2^{(m+n)}$ in general, so it is intractable for realistic RBMs; this is why sampling methods such as contrastive divergence are used.

b) Compute E(v, h) for v = (1, 0, 1) and h = (1, 0).

Ans: $a^Tv = 0.1(1) + 0(0) + (-0.1)(1) = 0$; $b^Th = (-0.5)(1) + 0.2(0) = -0.5$;

v^TW = row 1 + row 3 = (1.0 − 1.0, −0.5 + 0.5) = (0, 0), so $v^TWh = 0$.

$E(v, h) = -a^Tv - b^Th - v^TWh = -0 - (-0.5) - 0 =$ **0.5**.

c) Compute $P(h_1 = 1 \mid v)$ and $P(h_2 = 1 \mid v)$ using the sigmoid function.

Ans: $P(h_j = 1 \mid v) = \sigma(b_j + \Sigma_i v_i W_{ij})$, with $\sigma(x) = 1/(1 + e^{-x})$.

h_1: −0.5 + (1.0 + (−1.0)) = −0.5 ⇒ **$P(h_1 = 1 \mid v) = \sigma(-0.5) = 0.3775$**

h_2: 0.2 + (−0.5 + 0.5) = 0.2 ⇒ **$P(h_2 = 1 \mid v) = \sigma(0.2) = 0.5498$**

d) Explain why the bipartite structure of an RBM makes the hidden units conditionally independent given v and enables efficient block Gibbs sampling.

Ans: In an RBM there are connections only between visible and hidden units (bipartite graph), none inside a layer. Hence the energy splits into a sum of terms that each involve a single hidden unit: $E = -a^T v - \Sigma_j h_j(b_j + \Sigma_i v_i W_{ij})$, so $P(h \mid v) = \Pi_j P(h_j \mid v)$: the hidden units are conditionally independent given v (and likewise the visible units given h).

Therefore all hidden units can be sampled **in parallel** in one step, then all visible units in one step, alternately (block Gibbs sampling). This is much cheaper than updating units one by one as would be necessary in a fully connected Boltzmann machine.

Evidence Expected: *Probability expression, energy value, conditional probabilities and structural reasoning.*

TASK 8 – Restricted Boltzmann Machines: Contrastive Divergence | 4 Marks

Continue with the RBM of Task 7, the data vector v = (1, 0, 1) and the sampled hidden state h = (1, 0). Use learning rate ε = 0.1 and one step of contrastive divergence (CD-1).

a) Compute the reconstruction probabilities $P(v_i = 1 \mid h)$ for i = 1, 2, 3.

Ans: $P(v_i = 1 \mid h) = \sigma(a_i + \Sigma_j W_{ij} h_j)$ with h = (1, 0):

i = 1: 0.1 + 1.0 = 1.1 ⇒ **$\sigma(1.1) = 0.7503$**

i = 2: 0 + 0.5 = 0.5 ⇒ **$\sigma(0.5) = 0.6225$**

i = 3: −0.1 + (−1.0) = −1.1 ⇒ **$\sigma(-1.1) = 0.2497$**

b) Suppose the sampled reconstruction is v′ = (1, 1, 0). Compute $P(h'_j = 1 \mid v')$ for both hidden units.

Ans: With v′ = (1, 1, 0): $P(h'_j = 1 \mid v') = \sigma(b_j + \Sigma_i v'_i W_{ij})$.

h'_1: −0.5 + (1.0 + 0.5 + 0) = 1.0 ⇒ **$\sigma(1.0) = 0.7311$**

h'_2: 0.2 + (−0.5 + 1.0 + 0) = 0.7 ⇒ **$\sigma(0.7) = 0.6682$**

c) Using $\Delta W_{ij} = \varepsilon(v_i h_j - v'_i P(h'_j = 1 \mid v'))$, compute ΔW_{11} and ΔW_{31}.

Ans: $\Delta W_{11} = \varepsilon(v_1 h_1 - v'_1 P(h'_1 = 1 \mid v')) = 0.1(1 \times 1 - 1 \times 0.7311) = 0.1 \times 0.2689 =$ **0.0269**

$\Delta W_{31} = \varepsilon(v_3 h_1 - v'_3 P(h'_1 = 1 \mid v')) = 0.1(1 \times 1 - 0 \times 0.7311) = 0.1 \times 1 =$ **0.1000**

d) Explain why the exact gradient of the log-likelihood is intractable and how CD-1 approximates it.

Ans: The gradient of the log-likelihood is $\partial \log p(v)/\partial W_{ij} = \langle v_i h_j \rangle_data - \langle v_i h_j \rangle_model$. The model expectation needs a sum over all 2^(m+n) joint states (it depends on the partition function Z) or running a Gibbs chain to equilibrium, which is intractable for realistic sizes.

CD-1 replaces the model expectation by a one-step reconstruction: start the chain at the data vector v, sample $h \sim P(h \mid v)$, reconstruct $v' \sim P(v \mid h)$, compute $P(h' \mid v')$, and update $\Delta W_{ij} = \varepsilon(v_i h_j - v'_i P(h'_j = 1 \mid v'))$. It is a cheap, slightly biased approximation that works well in practice.

Evidence Expected: *Reconstruction probabilities, hidden probabilities, weight updates and explanation of contrastive divergence.*

TASK 9 – Auto Encoders: Architecture and Reconstruction | 4 Marks

An auto encoder compresses a scan into a code and reconstructs it. A 784–128–784 auto encoder is proposed for the 28 × 28 scans. To study the computation by hand, a small 2–1–2 auto encoder is used: the input is x = (2, 4), the single hidden neuron uses ReLU with weights (0.5, 0.25) and bias 0, and the linear output layer uses weights (0.8, 1.6) with zero bias.

a) For the 784–128–784 auto encoder, calculate the number of trainable parameters (including biases) and the compression ratio.

Ans: Encoder: 784 × 128 + 128 = 100,480. **Decoder:** 128 × 784 + 784 = 101,136. **Total = 201,616 trainable parameters.**

Compression ratio = 784 / 128 = **6.125 : 1** (each scan is stored as a 128-number code).

b) For the 2–1–2 auto encoder, compute the code h and the reconstruction $\hat{x}$, and calculate the loss $L = \frac{1}{2} \Sigma (x_i - \hat{x}_i)^2$.

Ans: Code: $h = \text{ReLU}(0.5 \times 2 + 0.25 \times 4 + 0) = \text{ReLU}(2) = \mathbf{2}$.

Reconstruction: $\hat{x} = (0.8 \times 2, 1.6 \times 2) = \mathbf{(1.6, 3.2)}$.

Loss: $L = \frac{1}{2}[(2 - 1.6)^2 + (4 - 3.2)^2] = \frac{1}{2}[0.16 + 0.64] = \mathbf{0.40}$.

c) Explain what can happen if the hidden layer is larger than the input without any constraint, and how an undercomplete bottleneck prevents it.

Ans: If the hidden layer is larger than the input and there is no constraint, the auto encoder can simply learn the **identity mapping** (copy the input to the output), reaching zero reconstruction error without learning any useful structure (and it would even copy noise).

An **undercomplete bottleneck** (code smaller than the input, e.g. 128 < 784) cannot pass every pixel through, so the network is forced to compress and keep only the most informative, regular patterns of the data; this makes the code a useful representation.

d) State the relationship between a linear auto encoder trained with squared error and Principal Component Analysis.

Ans: A linear auto encoder (linear activations) trained with squared error learns a code that spans the **same subspace as the top-k principal components (PCA)** of the data, i.e. the subspace of maximum variance, which gives the minimum reconstruction error (the weights need not be the orthonormal PCs themselves). Non-linear auto encoders generalise PCA to non-linear manifolds.

Evidence Expected: *Parameter count, forward-pass calculation, loss value and conceptual reasoning.*

TASK 10 – Auto Encoder Variants | 3 Marks

The hospital considers a denoising auto encoder and a sparse auto encoder. For one scan, the reconstruction loss is 0.40, the hidden activations are h = (0.8, 0, 0.1, 0) and the sparsity weight is $\lambda = 0.5$.

a) Write the training objective of a denoising auto encoder and explain how corrupting the input helps the network learn robust features.

Ans: Denoising auto encoder objective: corrupt the input, $\tilde{x} \sim q(\tilde{x} \mid x)$ (e.g. add Gaussian noise or randomly set pixels to zero), and train the network to reconstruct the **clean** input:

$L = E_\{x, \tilde{x}\} \, \|x - g(f(\tilde{x}))\|^2$

Because the target is the clean scan, the network cannot just copy its input; it has to learn the dependencies between pixels and the structure of real scans in order to remove the noise or fill in missing parts. The learned features are therefore robust to noise and the identity solution is avoided.

b) For the sparse auto encoder with an L1 penalty, compute the total loss = reconstruction loss + $\lambda \Sigma |h_j|$.

Ans: $\Sigma|h_j| = 0.8 + 0 + 0.1 + 0 = 0.9$; $\lambda\Sigma|h_j| = 0.5 \times 0.9 = 0.45$.

Total loss = 0.40 + 0.45 = 0.85.

c) Choose the better variant for noisy scans from older machines and justify the choice. **Evidence**

Expected: *Objective functions, numerical loss and a justified choice of variant.*

Ans: The **denoising auto encoder** is the better choice for noisy scans from older machines: it is explicitly trained to map noisy inputs to clean outputs, so it learns noise-robust features and removes noise during reconstruction. The sparse auto encoder only forces few active hidden units; it still tries to reconstruct the noisy input itself and so is not specifically robust to noise. (A sparsity penalty can additionally be combined with denoising if wanted.)

TASK 11 – Deep Learning Applications | 4 Marks

The hospital wants to use deep learning in three ways: classifying scans, detecting unusual scans with an auto encoder trained only on normal scans, and extracting findings from radiologist text reports. For the detector, normal validation scans give reconstruction errors with mean 0.03 and standard deviation 0.01. A scan is flagged as unusual if its error exceeds $\mu + 3\sigma$. Four new scans have errors 0.02, 0.05, 0.07 and 0.31.

a) Match each of the three uses to a suitable deep learning architecture and justify each choice.

Ans: Classifying scans → Convolutional Neural Network (CNN): convolution with shared weights exploits local spatial structure, is translation-tolerant and needs far fewer parameters than a fully connected network on 28 × 28 or larger images.

Detecting unusual scans → Auto encoder (undercomplete/convolutional) trained on normal scans only: scans that it cannot reconstruct well (high reconstruction error) are unusual.

Extracting findings from radiologist reports → sequence model for text: an LSTM/GRU recurrent network or a Transformer, which reads word sequences and context (named-entity/finding extraction).

b) Calculate the threshold and state which of the four scans are flagged.

Ans: Threshold = $\mu + 3\sigma = 0.03 + 3(0.01) =$ **0.06**.

Errors: 0.02 (≤ 0.06, not flagged), 0.05 (not flagged), 0.07 (> 0.06, **flagged**), 0.31 (> 0.06, **flagged**). So the scans with errors **0.07 and 0.31** are flagged as unusual.

c) Explain why only normal scans are used to train the detector.

Ans: Abnormal scans are rare, very varied, and often unlabelled or unknown in advance. Training only on normal scans lets the auto encoder learn the manifold of normal anatomy, so normal scans give low error while any scan that differs (including types of abnormality never seen before) is reconstructed poorly and gets a high error. It needs no labelled abnormal examples. If abnormal scans were included, the network would learn to reconstruct them too and the error would no longer separate them.

d) Discuss two practical concerns, such as privacy, interpretability or bias, in deploying deep learning in healthcare.

Ans: 1. Privacy: scans and reports are sensitive patient data. They must be de-identified, access controlled, and used with consent and in line with regulations (e.g. HIPAA/GDPR/DPDP Act); techniques such as federated learning or encryption reduce the risk of leakage.

2. Interpretability: a deep network is a black box, but radiologists must understand and trust a flag. Saliency maps/Grad-CAM, reconstruction-error maps and a human-in-the-loop review are needed.

(Bias is also a concern: a model trained on data from limited hospitals, devices or populations may perform worse on others, so it must be validated across groups and sites.)

Evidence Expected: *Architecture matching, threshold calculation, reasoning on training data and deployment concerns.*

TASK 12 – Design Challenge: Selecting and Adapting a Deep Learning Architecture | 8 Marks

Changed Requirement

The hospital initially built a shallow network with one hidden layer of 1,000 sigmoid neurons, trained only on the 1,000 labelled 28 × 28 scans. It must now handle 200,000 unlabelled scans of 256 × 256 pixels collected from five hospitals, only 2,000 labelled scans, noisy images, detection of rare abnormalities and deployment on devices with limited memory. Decide whether to retain, modify or redesign the architecture.

A. Impact Analysis | 2 Marks

a) Explain how 256 × 256 inputs, noisy images and scarce labels affect the parameter count, training and generalisation of the original network.

Ans: Parameters: 256 × 256 = 65,536 inputs (about 84 times more than 784). A fully connected layer of 1,000 neurons then has 65,536 × 1,000 + 1,000 = **65,537,000 parameters** (about 262 MB in float32) against 785,000 before.

Generalisation/training: with only 2,000 labelled scans, 65.5 million parameters (≈ 32,800 per label) overfit severely, and a fully connected layer ignores the spatial structure of the image. **Noise:** noisy pixels feed directly into every neuron and sigmoid units saturate, so the network is sensitive to noise and training is slow. **Unlabelled data:** the 200,000 unlabelled scans are not used at all.

b) Discuss why representation learning, depth and activation behaviour become important.

Ans: Representation learning is needed because raw 65,536-pixel vectors are poor features; the 200,000 unlabelled scans can be used to learn features of normal anatomy and of the differences between the five hospitals' machines. **Depth** gives a hierarchy of features (edges → textures → structures → lesions) and, together with convolution/weight sharing, much better parameter efficiency. **Activation behaviour** matters because sigmoid has $\sigma' \leq 0.25$, which gives vanishing gradients in deeper networks, whereas ReLU/LReLU/ERELU keep gradients alive (LReLU/ERELU also avoid dead neurons).

B. Evaluation of the Original Approach | 2 Marks

a) Evaluate whether a single-hidden-layer sigmoid network is sufficient for the new requirement.

Ans: No, it is not sufficient. A single wide sigmoid layer cannot efficiently represent the hierarchical, local patterns of 256 × 256 scans or rare small abnormalities, would overfit the 2,000 labels, ignores the 200,000 unlabelled scans, is not noise-robust, and is too large for devices with limited memory. It must be redesigned.

b) Explain its limitations in terms of capacity, vanishing gradients, label scarcity and memory.

Ans: Capacity: high capacity in the wrong form (huge width, no hierarchy); it would need an enormous number of neurons to capture subtle lesions. **Vanishing gradients:** sigmoid derivatives are at most 0.25, so adding depth to fix capacity makes training stall. **Label scarcity:** about 65.5 million parameters for 2,000 labels. **Memory:** 65.5 million weights ≈ 262 MB in float32, and about 3–4 times that during training with gradients and optimizer states, which is too much for low-memory devices.

C. Adaptation or Redesign | 2 Marks

a) Propose a redesigned architecture using unsupervised pre-training (an auto encoder or RBM stack), a suitable depth and width, and a suitable activation function such as ReLU, LReLU or ERELU.

Ans: Stage 1 – unsupervised pre-training (200,000 unlabelled scans): a convolutional denoising auto encoder. Encoder: Conv 3×3 (16, stride 2) → Conv 3×3 (32, stride 2) → Conv 3×3 (64, stride

2) → Conv 3×3 (128, stride 2) → 1×1 Conv to 8 channels, giving a 16 × 16 × 8 = 2,048-value code (32 × compression of 65,536). The decoder mirrors the encoder (transposed convolutions).

Stage 2 – supervised network: keep the encoder, flatten the code → Dense 64 → 1 sigmoid output (normal / abnormal). The 4 depth levels give hierarchical features while the narrow convolutional layers keep the model small.

Activation: LReLU (slope 0.01) in all hidden layers: it is almost as cheap as ReLU, avoids dead neurons and vanishing gradients; ERELU is an alternative for extra noise robustness at the cost of computing exponentials. The decoder output is linear.

Parameters: encoder 160 + 4,640 + 18,496 + 73,856 + 1,032 = 98,184; classifier head 2,048 × 64 + 64 + 65 = 131,201; **total 229,385** (≈ 0.23 M, about 0.92 MB in float32, ≈ 286 times fewer than 65.5 M).

b) Explain how a denoising or sparse auto encoder, supervised fine-tuning and the size of the bottleneck code can be incorporated.

Ans: Denoising/sparse auto encoder: corrupt the input scans with Gaussian noise or masking and reconstruct the clean version, $L = \|x - g(f(\tilde{x}))\|^2$, which makes the features robust to noisy images from older machines; optionally add a sparsity term $\lambda\Sigma|h_j|$ on the code so that only a few units fire for each pattern, which helps to isolate rare abnormalities.

Bottleneck size: the code of 2,048 values is a 32 × compression. Too large a code lets the network copy its input; too small a code loses subtle lesions. It is chosen by comparing validation reconstruction error and downstream classification performance for several sizes (e.g. 512, 1,024, 2,048, 4,096).

Supervised fine-tuning: add the classification head, train it on the 2,000 labelled scans (first with the encoder frozen, then unfreeze with a small learning rate), using class-weighted cross-entropy for the rare abnormal class, data augmentation and early stopping. The decoder is discarded for deployment. The auto encoder's reconstruction error (threshold $\mu + 3\sigma$ on normal scans) can also flag rare abnormalities.

D. Evidence-Based Defence | 2 Marks

a) Propose experiments and evaluation measures to compare the redesigned approach with the original one.

Ans: Experiments: compare (1) the original 1-hidden-layer sigmoid network, (2) the same convolutional network trained from scratch, (3) the proposed pre-trained denoising auto encoder + fine-tuning. Ablations: with/without denoising, with/without sparsity, ReLU vs LReLU vs ERELU, bottleneck size. Use leave-one-hospital-out testing (generalization across the five hospitals), 5-fold cross-validation on the 2,000 labels, learning curves with 200/500/1,000/2,000 labels, and a robustness test with increasing added noise.

Measures: sensitivity (recall) for abnormal scans, specificity, precision, F1, ROC-AUC and PR-AUC (because abnormalities are rare), training-vs-validation error (generalization gap), reconstruction error/PSNR of the auto encoder, number of parameters, memory (MB) and inference time per scan on the target device.

b) Justify the final design using parameter count, generalisation, robustness to noise and deployment constraints.

Ans: Decision: redesign. Comparison of the two designs:

Criterion	Original (1 × 1,000 sigmoid)	Redesigned (conv. denoising AE + fine-tune)

Parameters	65,537,000 (≈ 262 MB, float32)	229,385 (≈ 0.92 MB; ≈ 0.23 MB with 8-bit quantization)
Parameters per label (2,000 labels)	≈ 32,800	≈ 115
Use of 200,000 unlabelled scans	None	Pre-training of the encoder
Noise robustness	Poor	Denoising objective + LReLU/ERELU
Gradient behaviour	Sigmoid, $\sigma' \leq 0.25$	LReLU, gradient does not vanish

Justification: (i) *Parameter count/generalization* – about 286 times fewer parameters and weight sharing greatly reduce variance, and unsupervised pre-training on 200,000 scans gives a good regularised starting point for 2,000 labels. (ii) *Robustness* – the denoising objective and smooth/leaky activations handle noisy images and multi-hospital differences. (iii) *Deployment* – under 1 MB of weights and only a forward pass of the encoder fit a memory-limited device. (iv) *Data characteristics* – the rare abnormalities are handled by class weighting, sparsity and reconstruction-error detection.

Evidence Expected: *Analysis of data scale and noise, a justified architecture, mathematical reasoning and a suitable evaluation plan.*

Final Mark Distribution – CO2

Task	**Topics Covered**	**Marks**
Task 1	Machine Learning and Deep Learning	3
Task 2	Representation Learning	4
Task 3	Width and Depth of Neural Networks	4
Task 4	Activation Functions – ReLU, LReLU and ERELU	5
Task 5	Dying Neurons and Choice of Activation	3
Task 6	Unsupervised Training of Neural Networks	3
Task 7	Restricted Boltzmann Machines – Energy and Inference	5
Task 8	Restricted Boltzmann Machines – Contrastive Divergence	4
Task 9	Auto Encoders – Architecture and Reconstruction	4
Task 10	Auto Encoder Variants	3
Task 11	Deep Learning Applications	4
Sub Total		**42**

Task 12	Design Challenge – Selecting and Adapting a Deep Learning Architecture	8
TOTAL		**50**